

ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВОЙ ПРОЕКТ

Тема: «Проектирование и разработка модуля бронирования оборудования»

ПО ПМ.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП-23/1
[Фамилия Имя Отчество]

Руководитель:
Трушина Александра Игоревна

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1 АНАЛИЗ ЗАДАЧИ БРОНИРОВАНИЯ ОБОРУДОВАНИЯ....5
1.1 Как используется общее оборудование....5
1.2 Обзор аналогов....5
1.3 Требования к модулю....6
1.4 Выбор средств разработки....7
Выводы по разделу 1....8
2 ПРОЕКТИРОВАНИЕ....9
2.1 Модель данных....9
2.2 Главное правило и граничные случаи....10
2.3 Состояния брони....11
2.4 Архитектура....13
2.5 Алгоритм создания брони....14
Выводы по разделу 2....15
3 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ....16
3.1 Структура проекта и каталог оборудования....16
3.2 Проверка доступности и создание брони....17
3.3 Отмена, текущие и завершённые брони....19
3.4 Хранение данных и развёртывание....21
3.5 Тестирование программного модуля....22
Выводы по разделу 3....25
ЗАКЛЮЧЕНИЕ....26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....27
ПРИЛОЖЕНИЕ А. ЛИСТИНГ АВТОМАТИЧЕСКИХ ТЕСТОВ....29
ПРИЛОЖЕНИЕ Б. ТАБЛИЦА РУЧНЫХ ПРОВЕРОК....40
ПРИЛОЖЕНИЕ В. ЛИСТИНГ КЛЮЧЕВЫХ КЛАССОВ....43
ПРИЛОЖЕНИЕ Г. ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ....49
ПРИЛОЖЕНИЕ Д. ИНСТРУКЦИЯ ЗАПУСКА....50

ВВЕДЕНИЕ

В колледже и в любой организации есть общее оборудование: проекторы, ноутбуки, фотоаппараты, микрофоны. Им пользуются разные сотрудники, и каждый раз возникает вопрос, свободен ли проектор на третью пару и кто взял ноутбук на выездное мероприятие. Пока бронирование ведётся устно, в общем чате или в бумажном журнале у лаборанта, двойные брони и споры «кто первый» неизбежны.

Актуальность темы определяется тем, что дорогостоящее оборудование должно использоваться без простоев и без конфликтов, а ответственный за него сотрудник – видеть полную картину: что занято, кем, на какой срок и что сейчас на обслуживании.

Степень разработанности. Задачи бронирования ресурсов решаются календарями общего доступа (Outlook, «Яндекс Календарь») и специализированными системами учёта, однако календари не знают о состоянии оборудования и не мешают создать пересекающуюся бронь, а специализированные системы избыточны для одного колледжа.

Объект исследования – процесс использования общего оборудования сотрудниками образовательной организации.

Предмет исследования – программный модуль бронирования оборудования с проверкой доступности на выбранный период.

Цель работы – разработать на платформе .NET 10 веб-модуль, который ведёт каталог оборудования, позволяет выбрать период использования, создавать и отменять брони, проверяет доступность оборудования и показывает текущие и завершённые бронирования.

Задачи:

– изучить процесс бронирования и сформулировать требования, главное правило и ограничения;

– спроектировать модель данных, состояния брони, архитектуру и алгоритм создания брони;

– реализовать каталог, проверку доступности, создание и отмену броней, списки и график загрузки;

– проверить модуль автоматическими тестами и ручными проверками;

– подготовить инструкции для запуска и для пользователя.

Методы: анализ предметной области, моделирование данных и состояний, проектирование алгоритма, модульное тестирование, функциональное тестирование в браузере.

Теоретическая значимость – формализация условия пересечения периодов и граничных случаев бронирования. Практическая значимость – модуль готов к работе в колледже: запускается одной командой, данные хранятся в одном файле SQLite.

Работа состоит из введения, трёх разделов, заключения, списка источников и приложений.

1 АНАЛИЗ ЗАДАЧИ БРОНИРОВАНИЯ ОБОРУДОВАНИЯ

1.1 Как используется общее оборудование

Оборудование колледжа числится на балансе как основные средства и имеет инвентарный номер, присвоенный при постановке на учёт [2]. За него отвечает материально ответственное лицо – лаборант или сотрудник ИТ-отдела. Преподаватели и сотрудники берут оборудование на время: на пару, на мероприятие, на выездную работу приёмной комиссии.

Без единой системы процесс выглядит так: сотрудник звонит или пишет ответственному, тот смотрит в тетрадь или вспоминает, свободен ли нужный проектор, и записывает новую бронь. Из этого вытекают типичные проблемы:

– две брони одного проектора на пересекающееся время – обнаруживается, когда оба преподавателя пришли за проектором;

– бронь оборудования, которое находится в ремонте или уже списано;

– отменённое мероприятие не освобождает время в журнале, и оборудование простаивает;

– нет истории: нельзя узнать, кто пользовался оборудованием накануне поломки.

Отсюда требование к модулю: решение о брони должно приниматься программой по данным о всех активных бронях и о состоянии оборудования, а брони не должны удаляться, чтобы сохранялась история.

1.2 Обзор аналогов

Сравнение возможных способов приведено в таблице 1.

Таблица 1 – Сравнение способов бронирования оборудования

Критерий	**Бумажный журнал**	**Общий календарь (Outlook, «Яндекс Календарь»)**	**Разрабатываемый модуль**
Запрет пересекающихся броней	нет	нет (только предупреждение у отдельных календарей)	да, проверка до сохранения
Учёт состояния оборудования	нет	нет	да: доступно, на обслуживании, списано
Поиск свободного оборудования на период	вручную	по каждому календарю отдельно	одна страница со списком свободного
История и причины отмен	частично	нет	журнал операций по каждой брони
Требуется	тетрадь	учётные записи у всех	браузер

1.3 Требования к модулю

Функции определены заданием: «Реализовать каталог оборудования, выбор периода использования, создание и отмену бронирования и проверку доступности оборудования. Предусмотреть просмотр текущих и завершённых бронирований» (таблица 2).

Таблица 2 – Функциональные требования

Код	**Функция**	**Содержание**
Т1	Каталог оборудования	поиск по названию, инвентарному номеру и категории; добавление, изменение; смена состояния вместо удаления
Т2	Выбор периода	начало и окончание с точностью до минуты; срок не больше 14 дней
Т3	Проверка доступности	страница «Свободно ли оборудование с … по …» со списком свободных и занятых единиц и причиной занятости
Т4	Создание брони	оборудование, сотрудник, период, цель; запрет пересечения с активными бронями
Т5	Отмена брони	только активной и ещё не закончившейся, с обязательной причиной
Т6	Текущие и завершённые	вкладки «Текущие» и «Завершённые и отменённые», фильтры по оборудованию, сотруднику, дате и статусу, автоматическое завершение
Т7	Справочник сотрудников	добавление и изменение

Главное правило: нельзя забронировать оборудование на период, который пересекается с другим активным бронированием этого же оборудования.

Ограничения: окончание позже начала; начало не в прошлом; срок не больше 14 дней; нельзя бронировать оборудование «На обслуживании» и «Списано»; отменить можно только активную бронь, которая ещё не закончилась; цель обязательна; брони не удаляются, оборудование не удаляется, а списывается; вывести оборудование на обслуживание или списать нельзя, пока у него есть текущие брони.

Нефункциональные требования: .NET 10, ASP.NET Core Razor Pages; SQLite; русский интерфейс и сообщения; неверные данные и неизвестные записи не приводят к сбою; логика отделена от страниц и покрыта тестами; время в логике берётся из заменяемого источника, чтобы правила с «прошлым» и «завершением» можно было проверять тестами.

1.4 Выбор средств разработки

Средства разработки приведены в таблице 3.

Таблица 3 – Средства разработки

Средство	Версия	Обоснование
C#, .NET SDK	14, 10.0	платформа с долгосрочной поддержкой, изучалась на занятиях
ASP.NET Core Razor Pages	10.0	каждая страница – пара «разметка + модель страницы»; для приложения, состоящего из форм и списков, проще, чем MVC [10]
Entity Framework Core, SQLite	10.0.15	хранение в одном файле, миграции, LINQ-запросы [11]
TimeProvider	входит в .NET	единый источник текущего времени, заменяемый в тестах [12]
Bootstrap	5.3.3	оформление страниц без собственного CSS-фреймворка [13]
xUnit	2.9.3	автоматические тесты
Git, GitHub Actions	–	история версий, сборка, тесты и ручные проверки на сервере

Razor Pages выбраны вместо MVC осознанно: в модуле нет общих действий, которые удобно группировать в контроллеры, – каждая страница (каталог, бронь, доступность, график) самостоятельна, и её код лежит рядом с разметкой. Для проверки HTTP-запросами добавлено несколько методов минимального API (Minimal API) в адресах /api/....

Выводы по разделу 1

Проанализирован процесс использования общего оборудования и выявлены проблемы: пересекающиеся брони, бронь неисправного оборудования, отсутствие истории. Сравнение с журналом и календарями показало, что нужен модуль, который учитывает состояние оборудования и запрещает пересечения. Сформулированы семь функциональных требований, главное правило и ограничения, выбраны Razor Pages на .NET 10, EF Core с SQLite и TimeProvider.

2 ПРОЕКТИРОВАНИЕ

2.1 Модель данных

Модель содержит четыре сущности (рисунок 1): оборудование (Equipment), сотрудник (Employee), бронь (Booking) и запись журнала (BookingHistory).

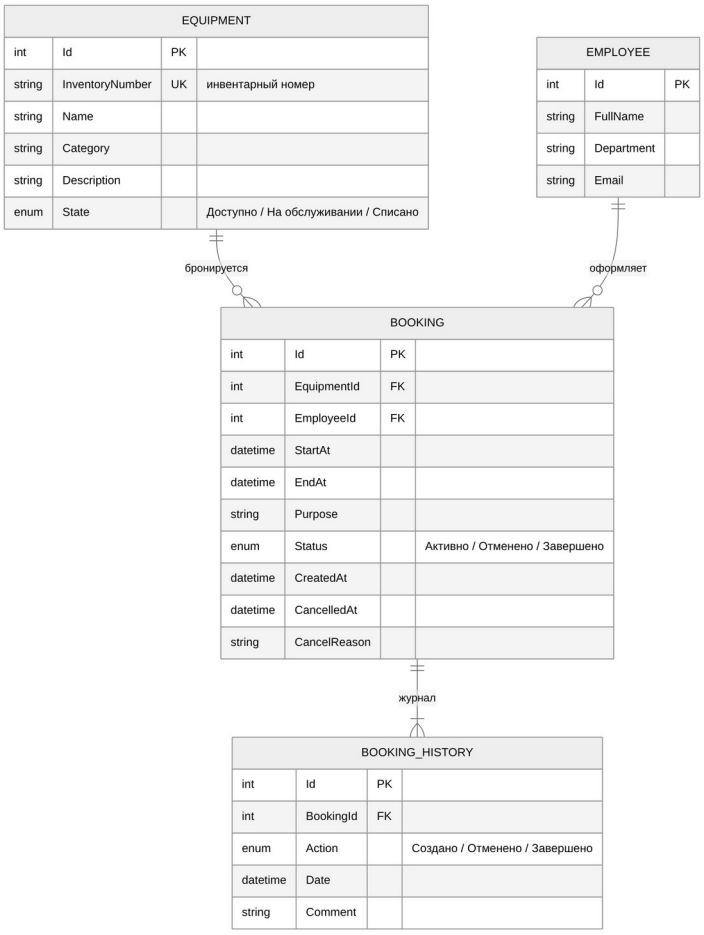


Рисунок 1 – ER-диаграмма модели данных

Таблица 4 – Сущности модели данных

Сущность	Поля	Ограничения
Equipment	Id, InventoryNumber, Name, Category, Description, State	инвентарный номер уникален; состояние: доступно, на обслуживании, списано
Employee	Id, FullName, Department, Email	ФИО и подразделение обязательны
Booking	Id, EquipmentId, EmployeeId, StartAt, EndAt, Purpose, Status, CreatedAt, CancelledAt, CancelReason	EndAt > StartAt; не удаляется; статус: активно, отменено, завершено
BookingHistory	Id, BookingId, Action, Date, Comment	создаётся при каждом изменении статуса брони

Описание сущностей приведено в таблице 4. Связи: оборудование 1 – N броней, сотрудник 1 – N броней, бронь 1 – N записей журнала. Для связей броней с оборудованием и сотрудниками задано правило Restrict: база не позволит удалить запись, на которую ссылаются брони. Для быстрой проверки пересечений создан составной индекс (EquipmentId, StartAt, EndAt).

2.2 Главное правило и граничные случаи

Новая бронь с периодом [Sн; Eн) пересекается с существующей бронью [Sс; Eс), если выполняется условие (1):

$$S_н < E_с \ \wedge \ E_н > S_с \qquad (1)$$

где Sн, Eн – начало и окончание новой брони;

Sс, Eс – начало и окончание существующей активной брони того же оборудования.

Периоды рассматриваются как полуинтервалы: момент окончания в бронь не входит. Поэтому бронь 12:00–14:00 после брони 10:00–12:00 допустима – условие Sн < Eс (12:00 < 12:00) ложно. Пересечение хотя бы на одну минуту (11:59–14:00) делает условие истинным, и бронь отклоняется. Отменённые и завершённые брони в проверке не участвуют.

Формула (1) реализована функцией BookingRules.Overlaps (листинг 1), а проверка самого периода – функцией ValidatePeriod. Обе функции «чистые»: не обращаются к базе и к часам, поэтому проверяются тестами напрямую.

Листинг 1 – Правило пересечения и проверка периода (Services/BookingRules.cs)

```
public static readonly TimeSpan MaxDuration = TimeSpan.FromDays(14);

public static bool Overlaps(DateTime newStart, DateTime newEnd, DateTime
existingStart, DateTime existingEnd)
{
    return newStart < existingEnd && newEnd > existingStart;
}

public static string? ValidatePeriod(DateTime start, DateTime end, DateTime
now)
{
    if (end <= start)
        return "Окончание брони должно быть позже начала";
    if (start < now.AddMinutes(-1))           // минута запаса на отправку
формы
        return "Нельзя бронировать на время в прошлом";
    if (end - start > MaxDuration)
        return $"Бронь не может быть длиннее {MaxDuration.TotalDays:0} дней";
    return null;                                  // период допустим
}
```

Примеры работы формулы при существующей брони 11:00–13:00 приведены в таблице 5.

Таблица 5 – Примеры проверки пересечения с бронью 11:00–13:00

Новый период	**Sн < Ec**	**Eн > Sc**	**Результат**
10:00–12:00	10:00 < 13:00 – да	12:00 > 11:00 – да	пересекается, отказ
13:00–15:00	13:00 < 13:00 – нет	15:00 > 11:00 – да	не пересекается, бронь допустима
09:00–11:00	09:00 < 13:00 – да	11:00 > 11:00 – нет	не пересекается, бронь допустима
12:59–15:00	12:59 < 13:00 – да	15:00 > 11:00 – да	пересечение на 1 минуту, отказ
11:30–12:30	11:30 < 13:00 – да	12:30 > 11:00 – да	вложена в бронь, отказ

Срок брони ограничен условием (2):

$$E_н - S_н \le 14 \text{ сут.} \qquad (2)$$

где $E_н - S_н$ – длительность новой брони.

2.3 Состояния брони

Бронь создаётся в статусе «Активно». Из него возможны два перехода (рисунок 2): отмена пользователем с указанием причины – в статус «Отменено» и

автоматическое завершение, когда время окончания прошло, – в статус «Завершено». Обратных переходов нет.

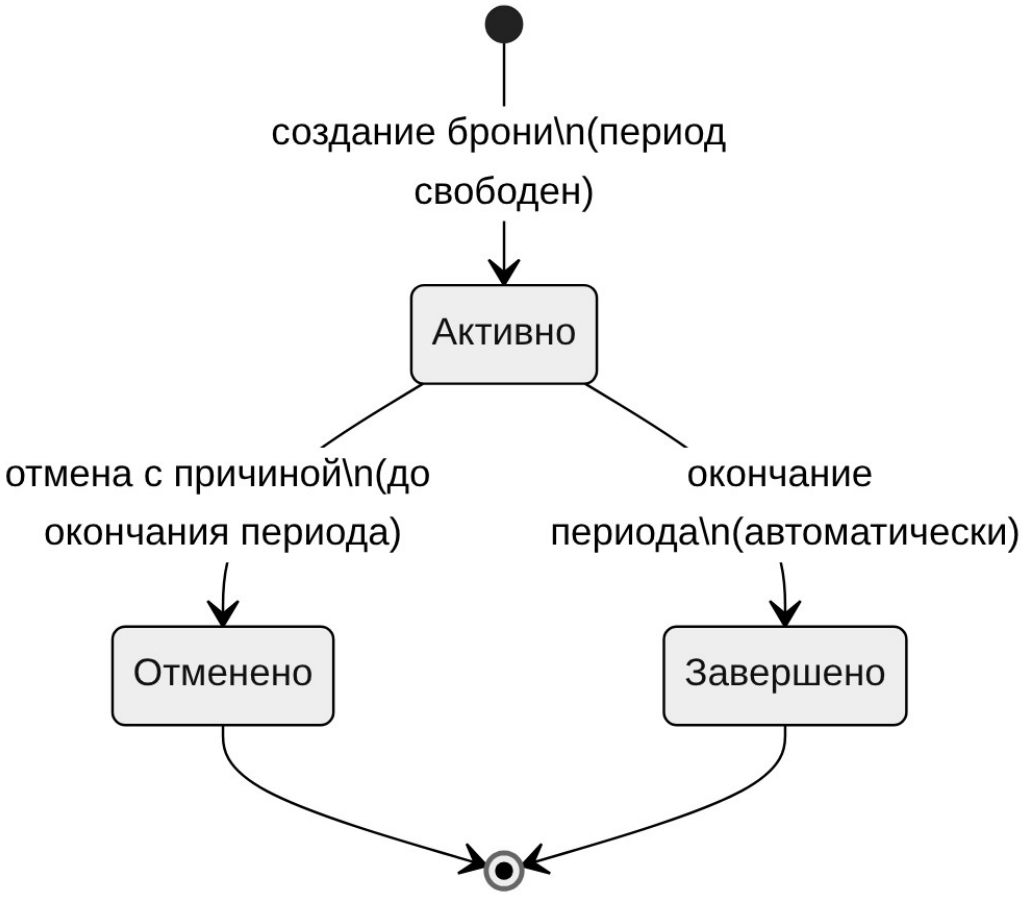


Рисунок 2 – Диаграмма состояний брони

Автоматическое завершение выполняет метод BookingService.CompleteExpired (листинг 2), который вызывается перед построением любого списка броней. Отдельная фоновая служба не нужна: статус обновляется в тот момент, когда пользователь на него смотрит.

Листинг 2 – Автоматическое завершение броней (Services/BookingService.cs)

```
public int CompleteExpired()
{
    var now = Now;                                          // время из
TimeProvider
    var expired = _db.Bookings
        .Where(b => b.Status == BookingStatus.Active && b.EndAt <= now)
        .ToList();
    foreach (var b in expired)
    {
        b.Status = BookingStatus.Completed;
        _db.BookingHistory.Add(new BookingHistory
        {
            BookingId = b.Id, Action = BookingAction.Completed, Date = now,
            Comment = "Завершено автоматически по окончании периода"
        });
    }
    if (expired.Count > 0) _db.SaveChanges();
    return expired.Count;
}
```

Пример: бронь на 13.11.2026 с 10:00 до 12:00. В 11:00 она остаётся активной (12:00 > 11:00), а при первом открытии списка после 12:00 получает статус «Завершено» и запись в журнале.

2.4 Архитектура

Архитектура модуля показана на рисунке 3. Страницы Razor Pages и методы API принимают запрос и вызывают сервисы; правила находятся в BookingService и в статическом классе BookingRules; доступ к данным – через AppDbContext.

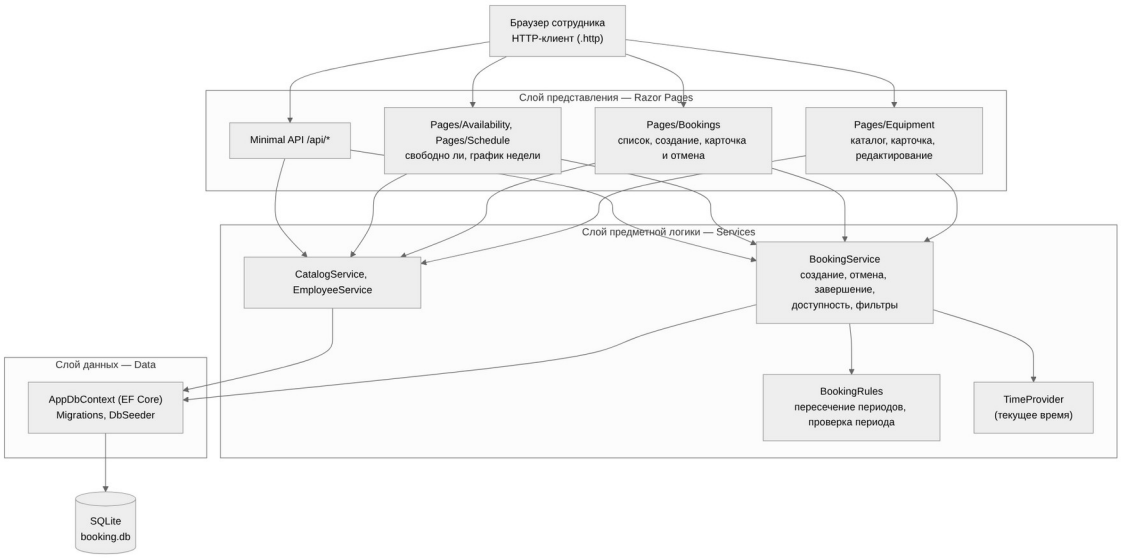


Рисунок 3 – Архитектура модуля

Особенность архитектуры – источник времени TimeProvider. Сервисы не обращаются к DateTime.Now напрямую, а получают текущее время у TimeProvider, зарегистрированного в контейнере зависимостей. В приложении это системные часы, а в тестах – «остановленные» часы FixedClock, которые тест сдвигает вперёд, чтобы проверить автоматическое завершение и запрет брони «в прошлом».

Сервисы возвращают объект OperationResult с видом ошибки: Invalid (неверные данные), NotFound (нет записи), Conflict (нарушено правило). Страница показывает текст ошибки над формой, а API превращает вид ошибки в код ответа 400, 404 или 409.

2.5 Алгоритм создания брони

Порядок проверок при создании брони показан на рисунке 4. Сначала выполняются проверки, не требующие базы (цель, период), затем проверяется оборудование и только потом ищется пересечение.

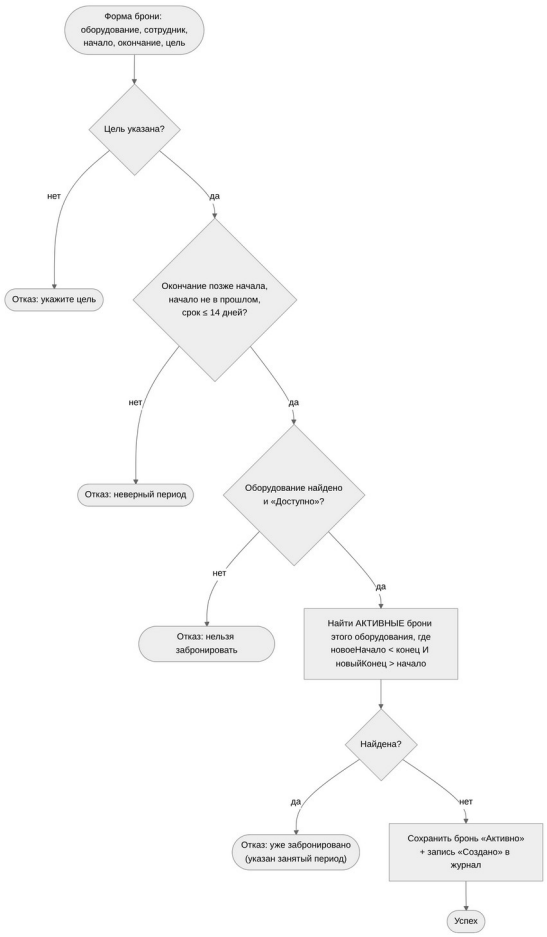


Рисунок 4 – Блок-схема алгоритма создания брони

Поиск пересечения и сохранение брони выполняются внутри критической секции lock. Если два сотрудника одновременно бронируют один проектор на одно время, второй запрос выполнит проверку после сохранения первого и получит отказ.

Выводы по разделу 2

Спроектирована модель из четырёх сущностей с журналом операций по каждой брони. Главное правило записано формулой (1) с явным разбором граничных случаев: касание концами допустимо, пересечение на минуту – нет. Определены три состояния брони и переходы между ними, в том числе автоматическое завершение. Выбрана трёхслойная архитектура с заменяемым источником времени, разработан алгоритм создания брони с защитой от одновременных запросов.

3 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ

3.1 Структура проекта и каталог оборудования

Назначение папок проекта приведено в таблице 6.

Таблица 6 – Структура проекта EquipmentBooking

Папка	Содержимое
Models	Equipment, Employee, Booking, BookingHistory, перечисления EquipmentState, BookingStatus, BookingAction
Data	AppDbContext, DbSeeder (учебные данные от даты первого запуска), Migrations
Services	BookingService, BookingRules, CatalogService, EmployeeService, OperationResult
Forms	BookingForm, BookingFilter, AvailabilityRow, WeekRow
Pages	Equipment, Bookings, Employees, Availability (доступность), Schedule (график), NotFound
EquipmentBooking.Tests	тесты xUnit: правила, сервис броней, поиск и каталог

Каталог (рисунок 5) выводится карточками с состоянием оборудования. Поиск выполняется по названию, инвентарному номеру и категории без учёта регистра, дополнительно доступны фильтры по категории и состоянию. Оборудование не удаляется: его можно вывести на обслуживание или списать, но только если у него нет текущих броней. Списанное оборудование в работу не возвращается.

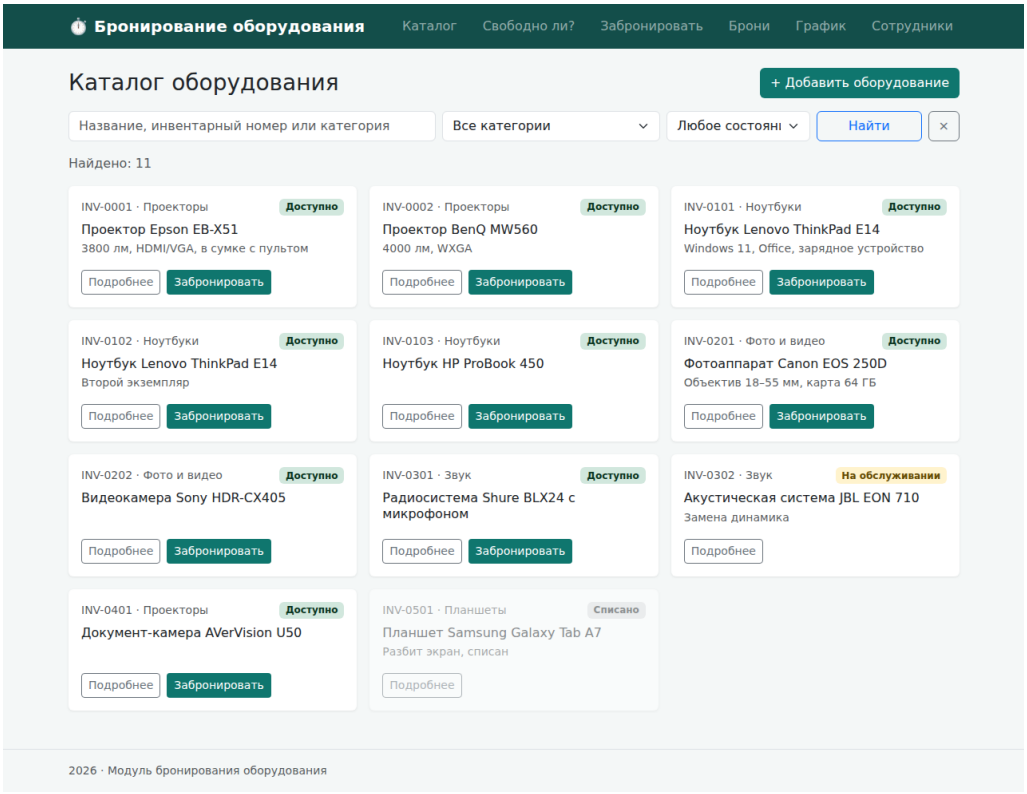


Рисунок 5 – Каталог оборудования

3.2 Проверка доступности и создание брони

Страница «Свободно ли?» (рисунок 6) принимает начало и окончание периода и делит каталог на свободное и недоступное оборудование. Для недоступного указана причина: состояние или конкретная пересекающаяся бронь с фамилией сотрудника. У каждой свободной единицы есть кнопка «Забронировать на этот период», которая открывает форму с уже заполненным периодом.

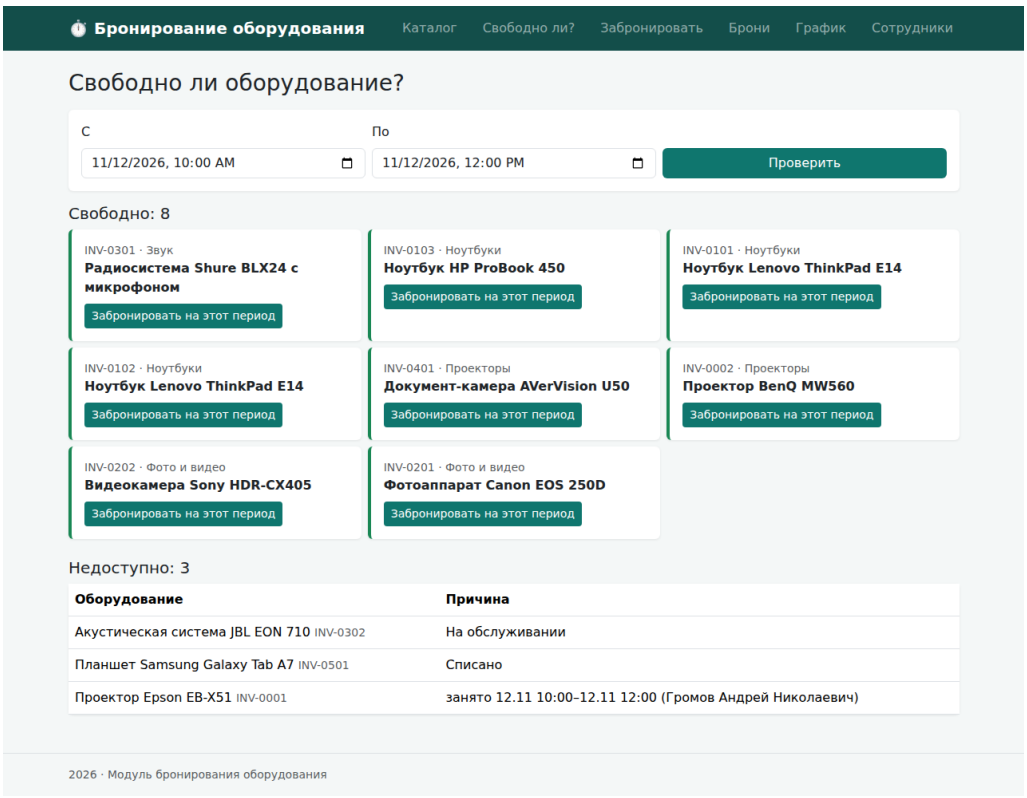


Рисунок 6 – Проверка доступности оборудования на период

Главное правило реализовано в методе BookingService.Create (листинг 3).

Листинг 3 – Поиск пересекающейся активной брони (Services/BookingService.cs, метод Create)

```
var conflict = _db.Bookings
    .Include(b => b.Employee)
    .Where(b => b.EquipmentId == form.EquipmentId && b.Status ==
BookingStatus.Active)
    .Where(b => form.StartAt < b.EndAt && form.EndAt > b.StartAt)
    .OrderBy(b => b.StartAt)
    .FirstOrDefault();
if (conflict != null)
    return OperationResult.Conflict($"«{equipment.Name}» уже забронировано с
{conflict.StartAt:dd.MM.yyyy HH:mm} ...");
```

Условие во втором Where – это формула (1). Сообщение об отказе называет занятый период и сотрудника, чтобы пользователь мог договориться или выбрать другое время (рисунок 7).

Пример из проверки: проектор INV-0001 забронирован на завтра с 10:00 до 12:00. При попытке забронировать его с 11:00 до 13:00 модуль ответил: ««Проектор Epson EB-X51» уже забронировано с 12.11.2026 10:00 по 12.11.2026 12:00 (Громов Андрей Николаевич). Выберите другое время или другое оборудование».

Модель страницы создания брони (листинг 4) только передаёт форму сервису и показывает результат: при успехе – переход на карточку новой брони, при ошибке – та же форма с сообщением.

Листинг 4 – Модель страницы создания брони (Pages/Bookings/Create.cshtml.cs, метод OnPost)

```
public IActionResult OnPost()
{
    if (ModelState.IsValid)                          // проверки атрибутов
формы
    {
        var result = _bookings.Create(Form);         // все правила — в
сервисе
        if (result.Success)
        {
            TempData["Message"] = "Бронь создана";
            return RedirectToPage("Details", new { id = result.CreatedId });
        }
        ModelState.AddModelError(string.Empty, result.Error!);
    }
    LoadLists();                                     // списки оборудования и
сотрудников
    return Page();                                   // форма с введёнными
данными и ошибкой
}
```

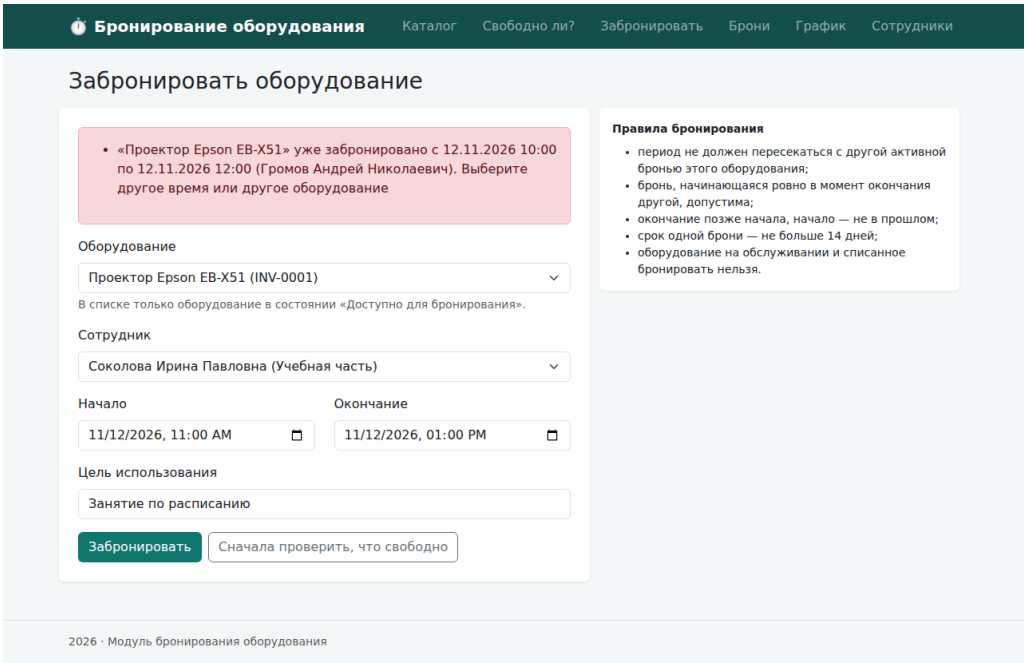


Рисунок 7 – Отклонение брони на занятый период

3.3 Отмена, текущие и завершённые брони

На карточке активной брони есть форма отмены с обязательной причиной. Метод отмены приведён в листинге 5.

Листинг 5 – Отмена брони с причиной (Services/BookingService.cs, метод Cancel)

```
public OperationResult Cancel(int bookingId, string? reason)
{
    var now = Now;
    var booking = _db.Bookings.FirstOrDefault(b => b.Id == bookingId);
    if (booking == null)
        return OperationResult.NotFound("Бронь не найдена");

    var error = BookingRules.CanCancel(booking, now);     // только активная
и не закончившаяся
    if (error != null)
        return OperationResult.Conflict(error);

    var text = (reason ?? "").Trim();
    if (text.Length < 3)
        return OperationResult.Invalid("Укажите причину отмены (не короче 3
символов)");

    booking.Status = BookingStatus.Cancelled;
    booking.CancelledAt = now;
    booking.CancelReason = text;
    _db.BookingHistory.Add(new BookingHistory
        { BookingId = booking.Id, Action = BookingAction.Cancelled, Date =
now, Comment = text });
    _db.SaveChanges();
    return OperationResult.Ok();
}
```

Пример: при повторной отмене той же брони через API сервер ответил «HTTP 409: Бронь уже отменена». После отмены статус становится «Отменено», сохраняются дата и причина, в журнал добавляется запись (рисунок 8). Отменённая бронь больше не мешает: на её время можно создать новую.

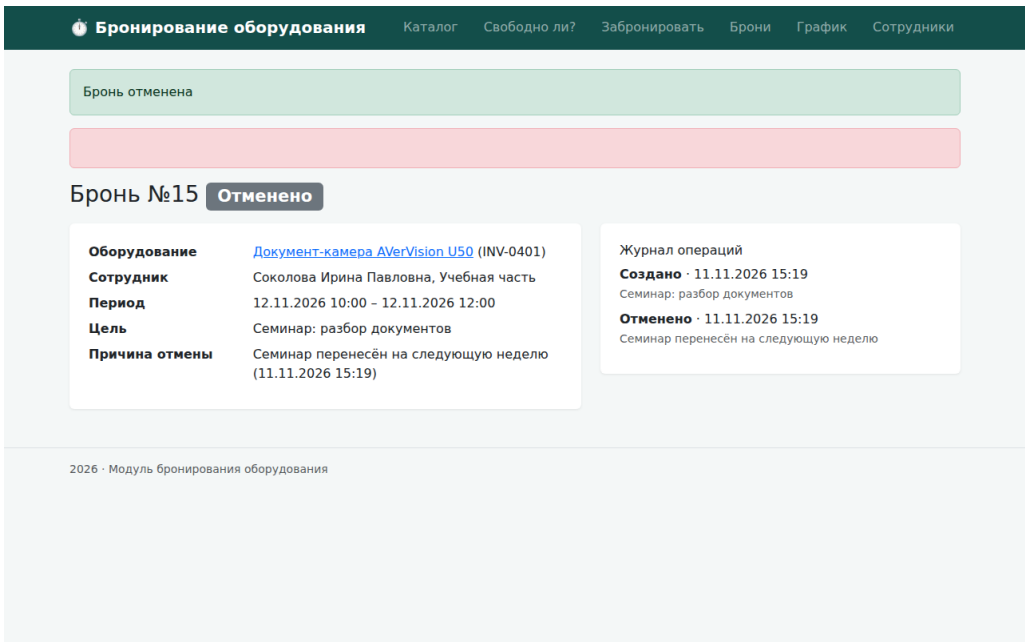


Рисунок 8 – Карточка отменённой брони с журналом операций

Список броней разделён на вкладки «Текущие» (активные, окончание ещё не наступило) и «Завершённые и отменённые». Фильтры по оборудованию, сотруднику, дате и статусу собираются в один запрос LINQ; фильтр по дате отбирает брони, которые хотя бы частично приходятся на выбранный день. Недельный график загрузки (рисунок 9) показывает занятые отрезки времени по каждой единице оборудования.

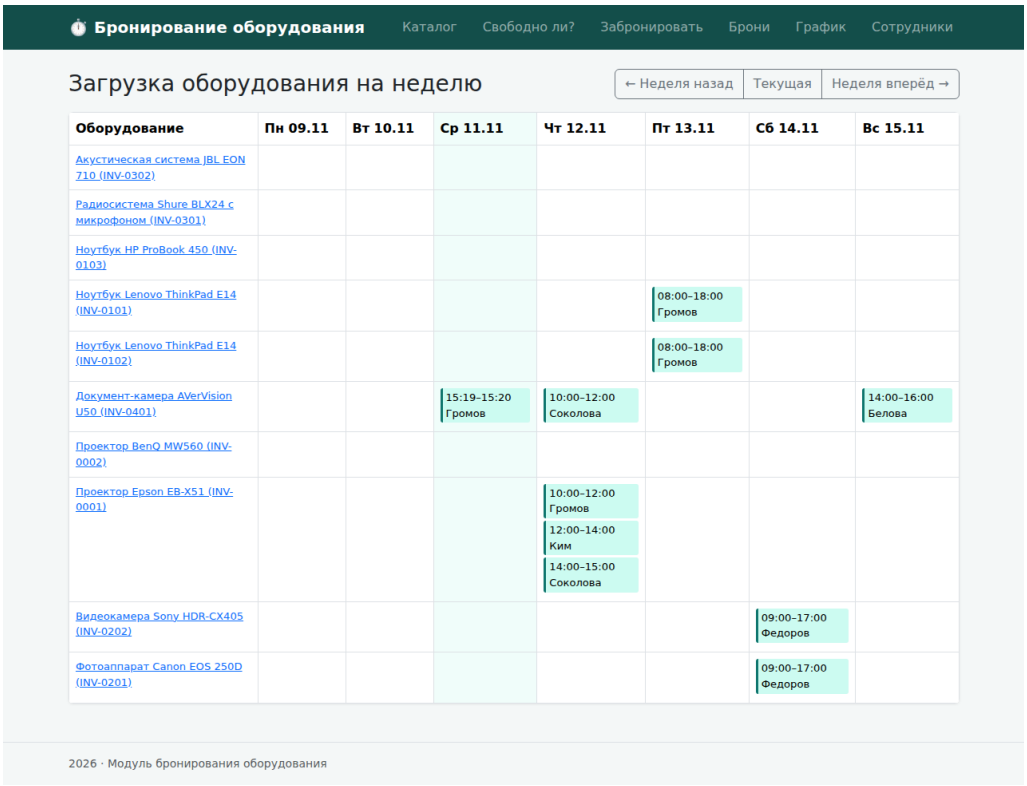


Рисунок 9 – Недельный график загрузки оборудования

3.4 Хранение данных и развёртывание

База SQLite создаётся миграцией InitialCreate при запуске (Database.Migrate()). При пустой базе DbSeeder добавляет 11 единиц оборудования (одна на обслуживании, одна списана), 8 сотрудников и 14 броней; даты броней отсчитываются от дня первого запуска, поэтому в учебных данных всегда есть и прошедшие, и предстоящие брони. Для возврата учебных данных достаточно удалить файл booking.db. Для развёртывания нужен только .NET SDK 10 (приложение Д).

3.5 Тестирование программного модуля

Подход. Автотесты проверяют логику без браузера: «чистые» функции BookingRules и сервисы на базе SQLite в памяти. Для каждого теста создаётся новая база и «остановленные» часы FixedClock (12.11.2026 09:00), поэтому результат не зависит от момента запуска. Интерфейс, перезапуск и развёртывание проверены вручную в браузере.

Тест граничного случая «касание концами» приведён в листинге 6: после брони 10:00–12:00 бронь 12:00–14:00 должна быть создана.

Листинг 6 – Тест «бронь начинается в момент окончания другой» (EquipmentBooking.Tests/BookingServiceTests.cs)

```
[Fact]
public void СоздатьБронь_НачинаетсяВМоментОкончанияДругой_Допустимо()
{
    Book(10, 12);                // подготовка: проектор занят завтра 10:00–
12:00

    var result = Book(12, 14);   // действие: бронь 12:00–14:00

    Assert.True(result.Success); // проверка: бронь создана
}
```

Состав тестов приведён в таблице 7, полный код – в приложении А.

Таблица 7 – Состав автоматических тестов

Класс	Что проверяется	Проверок	Результат
BookingRulesTests	формула пересечения на семи вариантах (касание с обеих сторон, 1 минута, вложение, накрытие); проверка периода, включая ровно 14 дней и 14 дней + 1 минута; возможность отмены в четырёх состояниях	16	пройдены
BookingServiceTests	создание; пересечение; касание; 1 минута; отменённая бронь не мешает; другое оборудование; обслуживание и списание; пустая цель; неверный период; неизвестные Id; отмена, повторная отмена, без причины, после окончания; автоматическое завершение до и после окончания	21	пройдены
SearchAndCatalogTests	текущие и завершённые; фильтры; доступность на период и на границе; поиск в каталоге; повтор инвентарного номера; списание при текущих бронях; возврат списанного; неизвестный Id; чтение новым контекстом	15	пройдены

Тесты запущены на сервере GitHub Actions (Ubuntu, .NET 10): всего 52, пройдено 52, не пройдено 0 (рисунок 10).

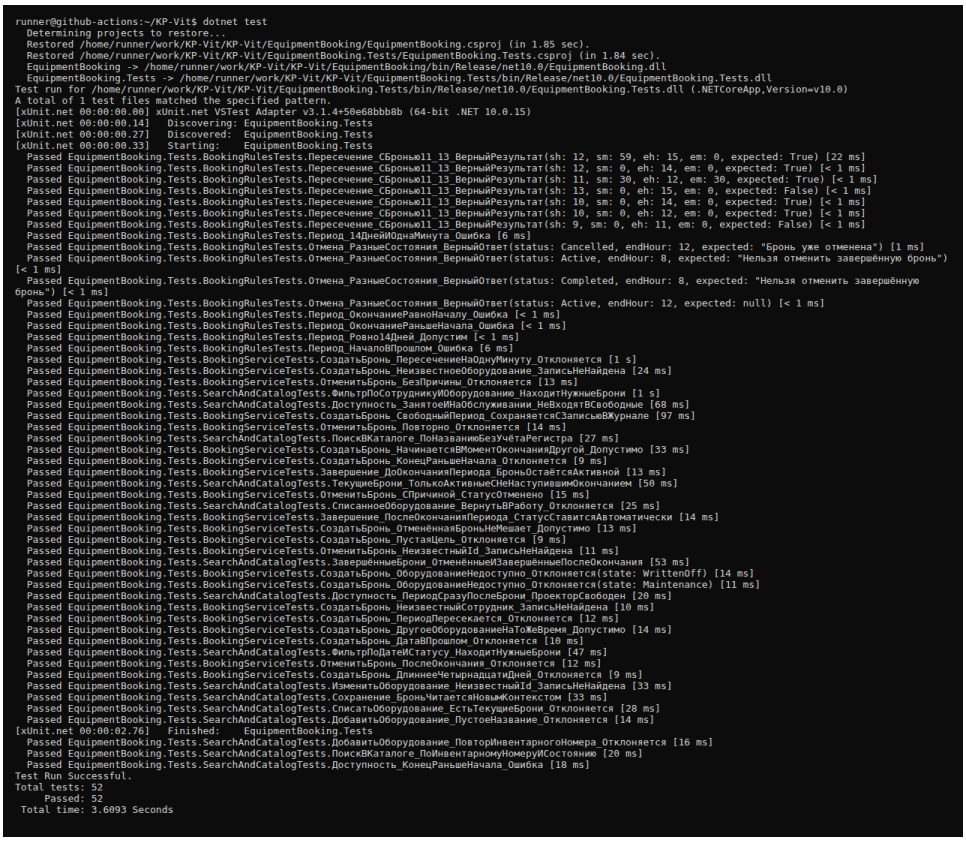


Рисунок 10 – Результаты выполнения автоматических тестов

Ручные проверки выполнены автоматизированным браузером по сценарию tools/screenshots.py на копии проекта, развёрнутой из архива: сценарий заполняет формы так же, как пользователь, и записывает сообщение, которое реально появилось на странице. Всего 31 проверка, основные приведены в таблице 8, полная таблица – в приложении Б.

Таблица 8 – Основные ручные проверки

№	Проверка	Ожидаемый результат	Фактический результат
8	создать бронь на свободный период	бронь создана	Бронь создана → Бронь №15 Активно
9	проектор 11:00–13:00 при брони 10:00–12:00	отказ	«Проектор Epson EB-X51» уже забронировано с 12.11.2026 10:00 по 12.11.2026 12:00 (Громов Андрей Николаевич). Выберите другое время или другое оборудование
11	14:00–15:00 сразу после 12:00–14:00	бронь создана	Бронь создана
19	отменить с причиной	статус «Отменено»	Бронь отменена → статус: Отменено
21	новая бронь на время отменённой	бронь создана	Бронь создана
22	бронь на 1 минуту, ожидание 70 с	статус «Завершено»	создана 201; через 70 с статус: Завершено; журнал: Создано / Завершено
29	обслуживание при текущих бронях	отказ	У оборудования есть текущие брони (3). Сначала отмените их
31	перезапуск приложения	данные на месте	INV-0601 найден: да; текущие брони: Найдено: 10 (до перезапуска: Найдено: 10)

Найденные при разработке ошибки и исправления приведены в таблице 9.

Таблица 9 – Найденные ошибки и исправления

№	Ошибка	Причина	Исправление	Повторная проверка
1	ошибка при отмене уже отменённой брони не показывалась на странице	сообщение выводилось внутри формы отмены, а форма у отменённой брони скрыта	сообщение перенесено над карточкой брони	проверка 20 (через API) и отмена на странице показывают текст ошибки
2	в файле решения у основного проекта был идентификатор, совпадающий с типом проекта	ошибка при ручном составлении .sln	присвоен уникальный идентификатор	решение собирается, тесты проходят
3	при запуске без миграции таблицы не создавались	Database.Migrate() без файлов миграции ничего не делает	создана миграция InitialCreate	база создаётся при чистом запуске (проверка на копии из архива)

Оставшиеся ограничения: нет входа в систему и ролей (заданием не предусмотрены); автоматическое завершение выполняется при открытии

страниц, а не по расписанию; защита от одновременных броней действует в пределах одного экземпляра приложения; стили Bootstrap загружаются из сети.

Выводы по тестированию: все 52 автоматические и 31 ручная проверка дали ожидаемый результат; главное правило и все граничные случаи из задания подтверждены тестами и проверками в браузере.

Выводы по разделу 3

Реализованы все функции задания: каталог с поиском и сменой состояния, проверка доступности на период, создание брони с проверкой пересечений, отмена с причиной, текущие и завершённые брони с фильтрами, автоматическое завершение, недельный график. Логика проверена 52 автотестами, интерфейс – 31 ручной проверкой на копии из архива.

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработан модуль бронирования оборудования. Цель достигнута; это подтверждается решением каждой задачи.

Процесс бронирования изучен в разделе 1: выявлены проблемы пересекающихся броней и брони неисправного оборудования, рассмотрены журнал и общие календари, сформулированы семь функций, главное правило и ограничения.

Проектирование выполнено в разделе 2: модель из четырёх сущностей с журналом операций, формула пересечения с разбором граничных случаев, три состояния брони, трёхслойная архитектура с заменяемым источником времени, алгоритм создания брони.

Реализация описана в подразделах 3.1–3.4: модуль на Razor Pages с каталогом, страницей доступности, созданием и отменой броней, вкладками текущих и завершённых броней, фильтрами и графиком загрузки.

Проверка выполнена в подразделе 3.5: 52 автоматических и 31 ручная проверка пройдены, найденные ошибки исправлены.

Инструкции подготовлены (приложения Г и Д).

Перспективы развития: вход в систему и роли «сотрудник» и «ответственный»; напоминания по электронной почте о начале брони; учёт выдачи и возврата оборудования с отметкой о состоянии; повторяющиеся брони (каждый вторник); фоновая служба автоматического завершения.

«___» _____ 2026 г. _____ / _____

(подпись) (ФИО)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : издание официальное : дата введения 1992-01-01. – Москва : Стандартинформ, 2010. – 22 с.

2. Об утверждении Федерального стандарта бухгалтерского учета ФСБУ 6/2020 «Основные средства» : Приказ Министерства финансов Российской Федерации от 17 сентября 2020 г. № 204н. – Текст : электронный // Официальный интернет-портал правовой информации. – URL: http://pravo.gov.ru (дата обращения: 11.11.2026).

3. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : межгосударственный стандарт : издание официальное : дата введения 2022-01-01. – Москва : Стандартинформ, 2021. – 15 с.

4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с.

5. Прайс, М. C# 10 и .NET 6. Современная кросс-платформенная разработка / М. Прайс. – Санкт-Петербург : Питер, 2023. – 848 с. [проверить]

6. Смит, Д. Entity Framework Core в действии / Д. Смит ; перевод с английского. – Москва : ДМК Пресс, 2022. – 690 с. [проверить]

7. Хориков, В. Принципы юнит-тестирования / В. Хориков. – Санкт-Петербург : Питер, 2021. – 320 с.

8. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с.

9. Lock, A. ASP.NET Core in Action / A. Lock. – 3rd ed. – Shelter Island : Manning, 2023. – 984 p. [проверить]

10. Razor Pages architecture and concepts in ASP.NET Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/razor-pages/ (дата обращения: 11.11.2026).

11. Entity Framework Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ef/core/ (дата обращения: 11.11.2026).

12. What is TimeProvider? – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/dotnet/standard/datetime/timeprovider-overview (дата обращения: 11.11.2026).

13. Bootstrap. Documentation. – Текст : электронный // Bootstrap. – URL: https://getbootstrap.com/docs/5.3/ (дата обращения: 11.11.2026).

Приложение А

Листинг автоматических тестов

Файл TestEnv.cs

```
using EquipmentBooking.Data;
using EquipmentBooking.Models;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace EquipmentBooking.Tests;

/// <summary>
/// «Остановленные» часы: время не идёт само, его двигает тест.
/// Так проверки «в прошлом» и «уже завершилась» не зависят от момента запуска
тестов.
/// </summary>
public class FixedClock : TimeProvider
{
    private DateTimeOffset _now;

    public FixedClock(DateTime now) => _now = new DateTimeOffset(now,
TimeSpan.Zero);

    public override DateTimeOffset GetUtcNow() => _now;
    public override TimeZoneInfo LocalTimeZone => TimeZoneInfo.Utc;

    public void Advance(TimeSpan span) => _now = _now.Add(span);
}

/// <summary>Тестовое окружение: пустая база SQLite в памяти и часы на
12.11.2026 09:00.</summary>
public sealed class TestEnv : IDisposable
{
    public static readonly DateTime Today = new(2026, 11, 12);
    private readonly SqliteConnection _connection;
    private readonly DbContextOptions<AppDbContext> _options;

    public FixedClock Clock { get; } = new(Today.AddHours(9));

    public TestEnv()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();
        _options = new
DbContextOptionsBuilder<AppDbContext>().UseSqlite(_connection).Options;
        using var db = Db();
        db.Database.EnsureCreated();
    }

    public AppDbContext Db() => new(_options);

    public Equipment AddEquipment(string inv = "INV-0001", string name =
"Проектор Epson EB-X51",
        EquipmentState state = EquipmentState.Available, string category =
"Проекторы")
    {
        using var db = Db();
        var e = new Equipment { InventoryNumber = inv, Name = name, Category =
category, State = state };
        db.Equipment.Add(e);
```

```
        db.SaveChanges();
        return e;
    }

    public Employee AddEmployee(string name = "Соколова Ирина Павловна")
    {
        using var db = Db();
        var e = new Employee { FullName = name, Department = "Учебная часть" };
        db.Employees.Add(e);
        db.SaveChanges();
        return e;
    }

    public void Dispose() => _connection.Dispose();
}
```

Файл BookingRulesTests.cs

```
using EquipmentBooking.Models;
using EquipmentBooking.Services;

namespace EquipmentBooking.Tests;

/// <summary>Правила без базы данных: пересечение периодов, проверка периода,
возможность отмены.</summary>
public class BookingRulesTests
{
    private static readonly DateTime D = TestEnv.Today;
    private static DateTime T(int h, int m = 0) => D.AddHours(h).AddMinutes(m);

    [Theory]
    [InlineData(10, 0, 12, 0, true)]   // 10:00–12:00 при брони 11:00–13:00 —
пересекается
    [InlineData(12, 0, 14, 0, true)]   // захватывает конец существующей
    [InlineData(13, 0, 15, 0, false)]  // начинается ровно в момент окончания —
допустимо
    [InlineData(9, 0, 11, 0, false)]   // заканчивается ровно в момент начала —
допустимо
    [InlineData(12, 59, 15, 0, true)]  // пересечение на 1 минуту (12:59–13:00)
    [InlineData(11, 30, 12, 30, true)] // целиком внутри
    [InlineData(10, 0, 14, 0, true)]   // целиком накрывает
    public void Пересечение_СБронью11_13_ВерныйРезультат(int sh, int sm, int eh,
int em, bool expected)
    {
        Assert.Equal(expected, BookingRules.Overlaps(T(sh, sm), T(eh, em),
T(11), T(13)));
    }

    [Fact]
    public void Период_ОкончаниеРаньшеНачала_Ошибка()
    {
        Assert.Equal("Окончание брони должно быть позже начала",
BookingRules.ValidatePeriod(T(12), T(10), T(9)));
    }

    [Fact]
    public void Период_ОкончаниеРавноНачалу_Ошибка()
    {
        Assert.NotNull(BookingRules.ValidatePeriod(T(12), T(12), T(9)));
    }

    [Fact]
    public void Период_НачалоВПрошлом_Ошибка()
    {
```

```
        Assert.Equal("Нельзя бронировать на время в прошлом",
BookingRules.ValidatePeriod(T(8), T(10), T(9)));
    }

    [Fact]
    public void Период_Ровно14Дней_Допустим()
    {
        Assert.Null(BookingRules.ValidatePeriod(T(10), T(10).AddDays(14),
T(9)));
    }

    [Fact]
    public void Период_14ДнейИОднаМинута_Ошибка()
    {
        Assert.NotNull(BookingRules.ValidatePeriod(T(10),
T(10).AddDays(14).AddMinutes(1), T(9)));
    }

    [Theory]
    [InlineData(BookingStatus.Active, 12, null)]
    [InlineData(BookingStatus.Cancelled, 12, "Бронь уже отменена")]
    [InlineData(BookingStatus.Completed, 8, "Нельзя отменить завершённую
бронь")]
    [InlineData(BookingStatus.Active, 8, "Нельзя отменить завершённую бронь")]
// окончание уже прошло
    public void Отмена_РазныеСостояния_ВерныйОтвет(BookingStatus status, int
endHour, string? expected)
    {
        var b = new Booking { Status = status, StartAt = T(7), EndAt =
T(endHour) };

        Assert.Equal(expected, BookingRules.CanCancel(b, T(9)));
    }
}
```

Файл BookingServiceTests.cs

```
using EquipmentBooking.Forms;
using EquipmentBooking.Models;
using EquipmentBooking.Services;
using Microsoft.EntityFrameworkCore;

namespace EquipmentBooking.Tests;

/// <summary>
/// Создание и отмена броней через сервис. Часы: 12.11.2026 09:00.
/// Во всех тестах бронируется завтрашний день (13.11.2026).
/// </summary>
public class BookingServiceTests : IDisposable
{
    private readonly TestEnv _env = new();
    private readonly Equipment _projector;
    private readonly Employee _teacher;
    private static readonly DateTime Tomorrow = TestEnv.Today.AddDays(1);

    public BookingServiceTests()
    {
        _projector = _env.AddEquipment();
        _teacher = _env.AddEmployee();
    }

    public void Dispose() => _env.Dispose();

    private OperationResult Book(int fromHour, int toHour, int? equipmentId =
null, string purpose = "Лекция",
```

```
            int fromMinute = 0, int? employeeId = null, DateTime? day = null)
        {
            var d = day ?? Tomorrow;
            using var db = _env.Db();
            return new BookingService(db, _env.Clock).Create(new BookingForm
            {
                EquipmentId = equipmentId ?? _projector.Id,
                EmployeeId = employeeId ?? _teacher.Id,
                StartAt = d.AddHours(fromHour).AddMinutes(fromMinute),
                EndAt = d.AddHours(toHour),
                Purpose = purpose
            });
        }

        private OperationResult Cancel(int id, string? reason)
        {
            using var db = _env.Db();
            return new BookingService(db, _env.Clock).Cancel(id, reason);
        }

        [Fact]
        public void СоздатьБронь_СвободныйПериод_СохраняетсяСЗаписьюВЖурнале()
        {
            var result = Book(10, 12);

            Assert.True(result.Success);
            using var db = _env.Db();
            var b = db.Bookings.Include(x => x.History).Single();
            Assert.Equal(BookingStatus.Active, b.Status);
            Assert.Equal(BookingAction.Created, Assert.Single(b.History).Action);
        }

        [Fact]
        public void СоздатьБронь_ПериодПересекается_Отклоняется()
        {
            Book(11, 13);

            var result = Book(10, 12);   // пример из задания: 10:00–12:00 при брони
11:00–13:00

            Assert.Equal(ErrorKind.Conflict, result.Kind);
            Assert.Contains("уже забронировано", result.Error);
        }

        [Fact]
        public void СоздатьБронь_НачинаетсяВМоментОкончанияДругой_Допустимо()
        {
            Book(10, 12);

            var result = Book(12, 14);

            Assert.True(result.Success);
        }

        [Fact]
        public void СоздатьБронь_ПересечениеНаОднуМинуту_Отклоняется()
        {
            Book(10, 12);

            var result = Book(11, 14, fromMinute: 59);   // 11:59–14:00

            Assert.Equal(ErrorKind.Conflict, result.Kind);
        }
```

```
    [Fact]
    public void СоздатьБронь_ОтменённаяБроньНеМешает_Допустимо()
    {
        var first = Book(10, 12);
        Cancel(first.CreatedId!.Value, "Занятие перенесено");

        var result = Book(10, 12);

        Assert.True(result.Success);
    }

    [Fact]
    public void СоздатьБронь_ДругоеОборудованиеНаТоЖеВремя_Допустимо()
    {
        var laptop = _env.AddEquipment("INV-0101", "Ноутбук Lenovo", category:
"Ноутбуки");
        Book(10, 12);

        var result = Book(10, 12, laptop.Id);

        Assert.True(result.Success);
    }

    [Theory]
    [InlineData(EquipmentState.Maintenance)]
    [InlineData(EquipmentState.WrittenOff)]
    public void СоздатьБронь_ОборудованиеНедоступно_Отклоняется(EquipmentState
state)
    {
        var broken = _env.AddEquipment("INV-0302", "Колонка JBL", state,
"Звук");

        var result = Book(10, 12, broken.Id);

        Assert.Equal(ErrorKind.Conflict, result.Kind);
        Assert.Contains("нельзя забронировать", result.Error);
    }

    [Fact]
    public void СоздатьБронь_ПустаяЦель_Отклоняется()
    {
        var result = Book(10, 12, purpose: "  ");

        Assert.Equal(ErrorKind.Invalid, result.Kind);
    }

    [Fact]
    public void СоздатьБронь_КонецРаньшеНачала_Отклоняется()
    {
        var result = Book(12, 10);

        Assert.Equal("Окончание брони должно быть позже начала", result.Error);
    }

    [Fact]
    public void СоздатьБронь_ДатаВПрошлом_Отклоняется()
    {
        var result = Book(7, 8, day: TestEnv.Today);   // сегодня 07:00, а
сейчас 09:00

        Assert.Equal("Нельзя бронировать на время в прошлом", result.Error);
    }

    [Fact]
```

```
    public void СоздатьБронь_ДлиннееЧетырнадцатиДней_Отклоняется()
    {
        using var db = _env.Db();
        var result = new BookingService(db, _env.Clock).Create(new BookingForm
        {
            EquipmentId = _projector.Id, EmployeeId = _teacher.Id, Purpose =
"Командировка",
            StartAt = Tomorrow.AddHours(9), EndAt =
Tomorrow.AddDays(15).AddHours(9)
        });

        Assert.Equal(ErrorKind.Invalid, result.Kind);
    }

    [Fact]
    public void СоздатьБронь_НеизвестноеОборудование_ЗаписьНеНайдена()
    {
        Assert.Equal(ErrorKind.NotFound, Book(10, 12, equipmentId: 999).Kind);
    }

    [Fact]
    public void СоздатьБронь_НеизвестныйСотрудник_ЗаписьНеНайдена()
    {
        Assert.Equal(ErrorKind.NotFound, Book(10, 12, employeeId: 999).Kind);
    }

    [Fact]
    public void ОтменитьБронь_СПричиной_СтатусОтменено()
    {
        var id = Book(10, 12).CreatedId!.Value;

        var result = Cancel(id, "Занятие перенесено");

        Assert.True(result.Success);
        using var db = _env.Db();
        var b = db.Bookings.Include(x => x.History).Single();
        Assert.Equal(BookingStatus.Cancelled, b.Status);
        Assert.Equal("Занятие перенесено", b.CancelReason);
        Assert.NotNull(b.CancelledAt);
        Assert.Contains(b.History, h => h.Action == BookingAction.Cancelled);
    }

    [Fact]
    public void ОтменитьБронь_Повторно_Отклоняется()
    {
        var id = Book(10, 12).CreatedId!.Value;
        Cancel(id, "Занятие перенесено");

        var result = Cancel(id, "Ещё раз");

        Assert.Equal(ErrorKind.Conflict, result.Kind);
        Assert.Equal("Бронь уже отменена", result.Error);
    }

    [Fact]
    public void ОтменитьБронь_БезПричины_Отклоняется()
    {
        var id = Book(10, 12).CreatedId!.Value;

        Assert.Equal(ErrorKind.Invalid, Cancel(id, "").Kind);
    }

    [Fact]
    public void ОтменитьБронь_НеизвестныйId_ЗаписьНеНайдена()
```

```
    {
        Assert.Equal(ErrorKind.NotFound, Cancel(999, "Причина").Kind);
    }

    [Fact]
    public void ОтменитьБронь_ПослеОкончания_Отклоняется()
    {
        var id = Book(10, 12).CreatedId!.Value;
        _env.Clock.Advance(TimeSpan.FromDays(2));   // бронь уже закончилась

        Assert.Equal(ErrorKind.Conflict, Cancel(id, "Поздно").Kind);
    }

    [Fact]
    public void Завершение_ПослеОкончанияПериода_СтатусСтавитсяАвтоматически()
    {
        Book(10, 12);

_env.Clock.Advance(TimeSpan.FromDays(1).Add(TimeSpan.FromHours(3)));   // 13.11
12:00

        using var db = _env.Db();
        int completed = new BookingService(db, _env.Clock).CompleteExpired();

        Assert.Equal(1, completed);
        using var db2 = _env.Db();
        var b = db2.Bookings.Include(x => x.History).Single();
        Assert.Equal(BookingStatus.Completed, b.Status);
        Assert.Contains(b.History, h => h.Action == BookingAction.Completed);
    }

    [Fact]
    public void Завершение_ДоОкончанияПериода_БроньОстаётсяАктивной()
    {
        Book(10, 12);

_env.Clock.Advance(TimeSpan.FromDays(1).Add(TimeSpan.FromHours(2)));   // 13.11
11:00 — бронь идёт

        using var db = _env.Db();
        Assert.Equal(0, new BookingService(db, _env.Clock).CompleteExpired());
    }
}
```

Файл SearchAndCatalogTests.cs

```
using EquipmentBooking.Forms;
using EquipmentBooking.Models;
using EquipmentBooking.Services;

namespace EquipmentBooking.Tests;

/// <summary>Текущие и завершённые брони, фильтры, доступность на период,
каталог, сохранение.</summary>
public class SearchAndCatalogTests : IDisposable
{
    private readonly TestEnv _env = new();
    private readonly Equipment _projector;
    private readonly Equipment _laptop;
    private readonly Employee _sokolova;
    private readonly Employee _gromov;
    private static readonly DateTime Tomorrow = TestEnv.Today.AddDays(1);

    public SearchAndCatalogTests()
    {
```

```
        _projector = _env.AddEquipment("INV-0001", "Проектор Epson EB-X51");
        _laptop = _env.AddEquipment("INV-0101", "Ноутбук Lenovo ThinkPad",
category: "Ноутбуки");
        _env.AddEquipment("INV-0302", "Колонка JBL EON",
EquipmentState.Maintenance, "Звук");
        _sokolova = _env.AddEmployee("Соколова Ирина Павловна");
        _gromov = _env.AddEmployee("Громов Андрей Николаевич");

        // проектор: завтра 10–12 (Соколова); ноутбук: завтра 14–16 (Громов),
отменённая бронь проектора 15–16
        Create(_projector.Id, _sokolova.Id, 10, 12);
        Create(_laptop.Id, _gromov.Id, 14, 16);
        var cancelled = Create(_projector.Id, _gromov.Id, 15, 16);
        using var db = _env.Db();
        new BookingService(db, _env.Clock).Cancel(cancelled, "Перенос");
    }

    public void Dispose() => _env.Dispose();

    private int Create(int equipmentId, int employeeId, int from, int to)
    {
        using var db = _env.Db();
        return new BookingService(db, _env.Clock).Create(new BookingForm
        {
            EquipmentId = equipmentId, EmployeeId = employeeId, Purpose =
"Занятие",
            StartAt = Tomorrow.AddHours(from), EndAt = Tomorrow.AddHours(to)
        }).CreatedId!.Value;
    }

    private List<Booking> Search(BookingFilter f)
    {
        using var db = _env.Db();
        return new BookingService(db, _env.Clock).Search(f);
    }

    [Fact]
    public void ТекущиеБрони_ТолькоАктивныеСНеНаступившимОкончанием()
    {
        Assert.Equal(2, Search(new BookingFilter { Tab = "current" }).Count);
    }

    [Fact]
    public void ЗавершённыеБрони_ОтменённыеИЗавершённыеПослеОкончания()
    {
        Assert.Single(Search(new BookingFilter { Tab = "finished" }));   // пока
только отменённая

        _env.Clock.Advance(TimeSpan.FromDays(2));   // обе активные брони
закончились

        Assert.Equal(3, Search(new BookingFilter { Tab = "finished" }).Count);
        Assert.Empty(Search(new BookingFilter { Tab = "current" }));
    }

    [Fact]
    public void ФильтрПоСотрудникуИОборудованию_НаходитНужныеБрони()
    {
        Assert.Equal(2, Search(new BookingFilter { EmployeeId =
_gromov.Id }).Count);
        Assert.Single(Search(new BookingFilter { EmployeeId = _gromov.Id,
EquipmentId = _laptop.Id }));
    }
```

```
    [Fact]
    public void ФильтрПоДатеИСтатусу_НаходитНужныеБрони()
    {
        Assert.Equal(3, Search(new BookingFilter { Date = Tomorrow }).Count);
        Assert.Empty(Search(new BookingFilter { Date = Tomorrow.AddDays(1) }));
        Assert.Single(Search(new BookingFilter { Status =
BookingStatus.Cancelled }));
    }

    [Fact]
    public void Доступность_ЗанятоеИНаОбслуживании_НеВходятВСвободные()
    {
        using var db = _env.Db();
        var (error, rows) = new BookingService(db,
_env.Clock).CheckAvailability(Tomorrow.AddHours(11), Tomorrow.AddHours(13));

        Assert.Null(error);
        var free = rows.Where(r => r.IsFree).Select(r =>
r.Equipment.Id).ToList();
        Assert.Equal(new[] { _laptop.Id }, free);                    //
проектор занят 10–12, колонка на обслуживании
        Assert.Contains("занято", rows.Single(r => r.Equipment.Id ==
_projector.Id).Reason);
    }

    [Fact]
    public void Доступность_ПериодСразуПослеБрони_ПроекторСвободен()
    {
        using var db = _env.Db();
        var (_, rows) = new BookingService(db,
_env.Clock).CheckAvailability(Tomorrow.AddHours(12), Tomorrow.AddHours(13));

        Assert.True(rows.Single(r => r.Equipment.Id == _projector.Id).IsFree);
// 12:00 — граница, отменённая 15–16 не мешает
    }

    [Fact]
    public void Доступность_КонецРаньшеНачала_Ошибка()
    {
        using var db = _env.Db();
        var (error, _) = new BookingService(db,
_env.Clock).CheckAvailability(Tomorrow.AddHours(13), Tomorrow.AddHours(11));

        Assert.NotNull(error);
    }

    [Fact]
    public void ПоискВКаталоге_ПоНазваниюБезУчётаРегистра()
    {
        using var db = _env.Db();
        var found = new CatalogService(db, _env.Clock).Search("ПРОЕКТОР");

        Assert.Equal(_projector.Id, Assert.Single(found).Id);
    }

    [Fact]
    public void ПоискВКаталоге_ПоИнвентарномуНомеруИСостоянию()
    {
        using var db = _env.Db();
        var catalog = new CatalogService(db, _env.Clock);

        Assert.Single(catalog.Search("inv-01"));
        Assert.Single(catalog.Search(null, EquipmentState.Maintenance));
    }
```

```
    [Fact]
    public void ДобавитьОборудование_ПовторИнвентарногоНомера_Отклоняется()
    {
        using var db = _env.Db();
        var result = new CatalogService(db, _env.Clock).Create(new Equipment
{ InventoryNumber = "inv-0001", Name = "Ещё проектор", Category =
"Проекторы" });

        Assert.Equal(ErrorKind.Conflict, result.Kind);
    }

    [Fact]
    public void ДобавитьОборудование_ПустоеНазвание_Отклоняется()
    {
        using var db = _env.Db();
        var result = new CatalogService(db, _env.Clock).Create(new Equipment
{ InventoryNumber = "INV-0900", Name = " ", Category = "Проекторы" });

        Assert.Equal(ErrorKind.Invalid, result.Kind);
    }

    [Fact]
    public void СписатьОборудование_ЕстьТекущиеБрони_Отклоняется()
    {
        using var db = _env.Db();
        var result = new CatalogService(db,
_env.Clock).ChangeState(_projector.Id, EquipmentState.WrittenOff);

        Assert.Equal(ErrorKind.Conflict, result.Kind);
        Assert.Contains("текущие брони", result.Error);
    }

    [Fact]
    public void СписанноеОборудование_ВернутьВРаботу_Отклоняется()
    {
        var old = _env.AddEquipment("INV-0501", "Планшет",
EquipmentState.WrittenOff, "Планшеты");
        using var db = _env.Db();

        var result = new CatalogService(db, _env.Clock).ChangeState(old.Id,
EquipmentState.Available);

        Assert.Equal("Списанное оборудование нельзя вернуть в работу",
result.Error);
    }

    [Fact]
    public void ИзменитьОборудование_НеизвестныйId_ЗаписьНеНайдена()
    {
        using var db = _env.Db();
        var result = new CatalogService(db, _env.Clock).Update(new Equipment
{ Id = 999, InventoryNumber = "INV-1", Name = "X", Category = "Y" });

        Assert.Equal(ErrorKind.NotFound, result.Kind);
    }

    [Fact]
    public void Сохранение_БроньЧитаетсяНовымКонтекстом()
    {
        using var db = _env.Db();
        var b = new BookingService(db, _env.Clock).GetById(1);

        Assert.NotNull(b);
```

```
        Assert.Equal("Проектор Epson EB-X51", b!.Equipment!.Name);
        Assert.Equal("Соколова Ирина Павловна", b.Employee!.FullName);
    }
}
```

Приложение Б

Таблица ручных проверок

Фактические результаты прочитаны со страниц приложения и из ответов API сценарием tools/screenshots.py на копии проекта, развёрнутой из архива на сервере GitHub Actions.

Таблица 10 – Ручные проверки (полная таблица)

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат
1	Главная	открыть главную страницу	сводка: доступно, текущие брони, на сегодня, на обслуживании	9 / 8 / 0 / 1
2	Каталог	открыть «Каталог»	11 единиц оборудования	карточек в каталоге: 11
3	Поиск	ввести «ноутбук»	три ноутбука	Ноутбук Lenovo ThinkPad E14; Ноутбук Lenovo ThinkPad E14; Ноутбук HP ProBook 450
4	Фильтр состояния	состояние «На обслуживании»	акустическая система JBL	Акустическая система JBL EON 710
5	Карточка оборудования	открыть INV-0001	брони проектора	броней в карточке INV-0001: 3
6	Доступность	завтра 10:00–12:00	списки «Свободно» и «Недоступно» с причинами	Свободно: 8 / Недоступно: 3
7	Доступность	окончание раньше начала	сообщение об ошибке	Окончание периода должно быть позже начала
8	Создание брони	документ-камера, завтра 10:00–12:00, цель указана	бронь создана, статус «Активно»	Бронь создана → Бронь №15 Активно
9	Главное правило	проектор INV-0001, завтра 11:00–13:00 (занят 10:00–12:00)	отказ с указанием занятого периода	«Проектор Epson EB-X51» уже забронировано с 12.11.2026 10:00 по 12.11.2026 12:00 (Громов Андрей Николаевич). Выберите другое время или другое оборудование

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат
10	Граница: 1 минута	проектор, 13:59–16:00 (занят 12:00–14:00)	отказ	«Проектор Epson EB-X51» уже забронировано с 12.11.2026 12:00 по 12.11.2026 14:00 (Ким Елена Викторовна). Выберите другое время или другое оборудование
11	Граница: касание	проектор, 14:00–15:00 сразу после 12:00–14:00	бронь создана	Бронь создана
12	Период	окончание раньше начала	отказ	Окончание брони должно быть позже начала
13	Период	сегодня 00:00–01:00 (в прошлом)	отказ	Нельзя бронировать на время в прошлом
14	Период	15 дней	отказ: не длиннее 14 дней	Бронь не может быть длиннее 14 дней
15	Цель	пустая цель	отказ	Укажите цель использования
16	Состояние	API: бронь акустической системы (на обслуживании)	409	HTTP 409: «Акустическая система JBL EON 710» нельзя забронировать: состояние «На обслуживании»
17	Состояние	API: бронь списанного планшета	409	HTTP 409: «Планшет Samsung Galaxy Tab A7» нельзя забронировать: состояние «Списано»
18	Отмена	отменить без причины	отказ	Укажите причину отмены (не короче 3 символов)
19	Отмена	отменить с причиной	статус «Отменено»	Бронь отменена → статус: Отменено
20	Отмена	API: повторная отмена	409 «Бронь уже отменена»	HTTP 409: Бронь уже отменена
21	Отменённая не мешает	снова забронировать документ-камеру на то же время	бронь создана	Бронь создана
22	Автозавершение	бронь на текущую минуту, через 70 с открыть карточку	статус «Завершено», запись в журнале	создана 201; через 70 с статус: Завершено; журнал: Создано / Завершено
23	Текущие брони	вкладка «Текущие»	активные с ненаступившим окончанием	Найдено: 10

№	Функция	Исходные данные и действия	Ожидаемый результат	Фактический результат
24	Завершённые	вкладка «Завершённые и отменённые»	завершённые и отменённые	Найдено: 8
25	Фильтры	сотрудник Громов + дата завтра	брони Громова на завтра	Найдено: 1
26	График	открыть «График»	загрузка по дням недели	строк оборудования: 10, занятых отрезков: 10
27	Каталог: добавление	INV-0601 «3D-принтер Bambu Lab A1»	сохранено	Сохранено / Броней нет. → 3D-принтер Bambu Lab A1 Доступно
28	Каталог: повтор номера	инвентарный номер «inv-0001»	отказ	Оборудование с инвентарным номером INV-0001 уже есть в каталоге
29	Смена состояния	проектор с текущими бронями → «На обслуживание»	отказ	У оборудования есть текущие брони (3). Сначала отмените их
30	Неизвестная запись	/Bookings/Details/ 999 и /api/equipment/999	404 без падения	HTTP 404: Запись не найдена (404); API: HTTP 404: Оборудование с Id = 999 не найдено
31	Сохранение	остановить и снова запустить приложение	добавленное оборудование и брони на месте	INV-0601 найден: да; текущие брони: Найдено: 10 (до перезапуска: Найдено: 10)

Приложение В

Листинг ключевых классов

Файл Services/BookingRules.cs

```
using EquipmentBooking.Models;

namespace EquipmentBooking.Services;

/// <summary>
/// Правила бронирования в виде «чистых» функций без базы данных — их удобно
проверять тестами.
/// </summary>
public static class BookingRules
{
    /// <summary>Максимальный срок одной брони.</summary>
    public static readonly TimeSpan MaxDuration = TimeSpan.FromDays(14);

    /// <summary>
    /// ГЛАВНОЕ ПРАВИЛО. Два периода пересекаются, если новоеНачало &lt;
существующийКонец
    /// и новыйКонец &gt; существующееНачало. Касание концами (12:00–14:00 после
10:00–12:00)
    /// пересечением не считается.
    /// </summary>
    public static bool Overlaps(DateTime newStart, DateTime newEnd, DateTime
existingStart, DateTime existingEnd)
    {
        return newStart < existingEnd && newEnd > existingStart;
    }

    /// <summary>Проверка самого периода. Возвращает текст ошибки или null, если
период допустим.</summary>
    public static string? ValidatePeriod(DateTime start, DateTime end, DateTime
now)
    {
        if (end <= start)
            return "Окончание брони должно быть позже начала";
        // Минута запаса: форма отправляется не мгновенно
        if (start < now.AddMinutes(-1))
            return "Нельзя бронировать на время в прошлом";
        if (end - start > MaxDuration)
            return $"Бронь не может быть длиннее {MaxDuration.TotalDays:0}
дней";
        return null;
    }

    /// <summary>Бронь «текущая», если она активна и её окончание ещё не
наступило.</summary>
    public static bool IsCurrent(Booking b, DateTime now) => b.Status ==
BookingStatus.Active && b.EndAt > now;

    /// <summary>Отменить можно только активную бронь, которая ещё не
закончилась.</summary>
    public static string? CanCancel(Booking b, DateTime now)
    {
        if (b.Status == BookingStatus.Cancelled)
            return "Бронь уже отменена";
        if (b.Status == BookingStatus.Completed || b.EndAt <= now)
            return "Нельзя отменить завершённую бронь";
        return null;
```

```
    }
}
```

Файл Services/BookingService.cs

```
using EquipmentBooking.Data;
using EquipmentBooking.Forms;
using EquipmentBooking.Models;
using Microsoft.EntityFrameworkCore;

namespace EquipmentBooking.Services;

/// <summary>
/// Бронирование: проверка доступности, создание, отмена, автоматическое
завершение,
/// текущие и завершённые брони, фильтры, недельный график загрузки.
/// </summary>
public class BookingService
{
    // Две брони одного оборудования, пришедшие одновременно, проверяются по
очереди:
    // иначе обе увидели бы свободный период и обе сохранились бы.
    private static readonly object CreateLock = new();

    private readonly AppDbContext _db;
    private readonly TimeProvider _clock;

    public BookingService(AppDbContext db, TimeProvider clock)
    {
        _db = db;
        _clock = clock;
    }

    /// <summary>Текущее время берём у TimeProvider — в тестах его можно
«остановить».</summary>
    public DateTime Now => _clock.GetLocalNow().DateTime;

    /// <summary>
    /// Активные брони, у которых время окончания уже прошло, получают статус
«Завершено».
    /// Вызывается перед выводом списков, поэтому статус всегда актуален.
    /// </summary>
    public int CompleteExpired()
    {
        var now = Now;
        var expired = _db.Bookings.Where(b => b.Status == BookingStatus.Active
&& b.EndAt <= now).ToList();
        foreach (var b in expired)
        {
            b.Status = BookingStatus.Completed;
            _db.BookingHistory.Add(new BookingHistory
            {
                BookingId = b.Id, Action = BookingAction.Completed, Date = now,
Comment = "Завершено автоматически по окончании периода"
            });
        }
        if (expired.Count > 0) _db.SaveChanges();
        return expired.Count;
    }

    /// <summary>Создать бронь, если оборудование доступно и свободно на весь
период.</summary>
    public OperationResult Create(BookingForm form)
    {
        var now = Now;
```

```
        var purpose = (form.Purpose ?? "").Trim();
        if (purpose.Length < 3)
            return OperationResult.Invalid("Укажите цель использования (не
короче 3 символов)");

        var periodError = BookingRules.ValidatePeriod(form.StartAt, form.EndAt,
now);
        if (periodError != null)
            return OperationResult.Invalid(periodError);

        var equipment = _db.Equipment.FirstOrDefault(e => e.Id ==
form.EquipmentId);
        if (equipment == null)
            return OperationResult.NotFound("Оборудование не найдено");
        if (equipment.State != EquipmentState.Available)
            return OperationResult.Conflict(
                $"«{equipment.Name}» нельзя забронировать: состояние
«{StateName(equipment.State)}»");

        if (!_db.Employees.Any(e => e.Id == form.EmployeeId))
            return OperationResult.NotFound("Сотрудник не найден");

        lock (CreateLock)
        {
            // ГЛАВНОЕ ПРАВИЛО: период не должен пересекаться с другой АКТИВНОЙ
бронью этого оборудования.
            // Отменённые и завершённые брони не мешают.
            var conflict = _db.Bookings
                .Include(b => b.Employee)
                .Where(b => b.EquipmentId == form.EquipmentId && b.Status ==
BookingStatus.Active)
                .Where(b => form.StartAt < b.EndAt && form.EndAt > b.StartAt)
                .OrderBy(b => b.StartAt)
                .FirstOrDefault();
            if (conflict != null)
                return OperationResult.Conflict(
                    $"«{equipment.Name}» уже забронировано с
{conflict.StartAt:dd.MM.yyyy HH:mm} по {conflict.EndAt:dd.MM.yyyy HH:mm} " +
                    $"({conflict.Employee?.FullName}). Выберите другое время или
другое оборудование");

            var booking = new Booking
            {
                EquipmentId = form.EquipmentId,
                EmployeeId = form.EmployeeId,
                StartAt = form.StartAt,
                EndAt = form.EndAt,
                Purpose = purpose,
                Status = BookingStatus.Active,
                CreatedAt = now
            };
            booking.History.Add(new BookingHistory { Action =
BookingAction.Created, Date = now, Comment = purpose });
            _db.Bookings.Add(booking);
            _db.SaveChanges();
            return OperationResult.Ok(booking.Id);
        }
    }

    /// <summary>Отменить бронь с указанием причины (вместо удаления).</summary>
    public OperationResult Cancel(int bookingId, string? reason)
    {
        var now = Now;
        var booking = _db.Bookings.FirstOrDefault(b => b.Id == bookingId);
```

```
        if (booking == null)
            return OperationResult.NotFound("Бронь не найдена");

        var error = BookingRules.CanCancel(booking, now);
        if (error != null)
            return OperationResult.Conflict(error);

        var text = (reason ?? "").Trim();
        if (text.Length < 3)
            return OperationResult.Invalid("Укажите причину отмены (не короче 3
символов)");

        booking.Status = BookingStatus.Cancelled;
        booking.CancelledAt = now;
        booking.CancelReason = text;
        _db.BookingHistory.Add(new BookingHistory { BookingId = booking.Id,
Action = BookingAction.Cancelled, Date = now, Comment = text });
        _db.SaveChanges();
        return OperationResult.Ok();
    }

    public Booking? GetById(int id)
    {
        CompleteExpired();
        return _db.Bookings.AsNoTracking()
            .Include(b => b.Equipment).Include(b => b.Employee).Include(b =>
b.History)
            .FirstOrDefault(b => b.Id == id);
    }

    /// <summary>
    /// Список броней с фильтрами. Вкладка «current» — активные с ещё не
наступившим окончанием,
    /// «finished» — завершённые и отменённые.
    /// </summary>
    public List<Booking> Search(BookingFilter f)
    {
        CompleteExpired();
        var now = Now;
        IQueryable<Booking> q = _db.Bookings.AsNoTracking().Include(b =>
b.Equipment).Include(b => b.Employee);

        if (f.Tab == "current")
            q = q.Where(b => b.Status == BookingStatus.Active && b.EndAt > now);
        else if (f.Tab == "finished")
            q = q.Where(b => b.Status == BookingStatus.Completed || b.Status ==
BookingStatus.Cancelled);

        if (f.EquipmentId != null) q = q.Where(b => b.EquipmentId ==
f.EquipmentId);
        if (f.EmployeeId != null) q = q.Where(b => b.EmployeeId ==
f.EmployeeId);
        if (f.Status != null) q = q.Where(b => b.Status == f.Status);
        if (f.Date != null)
        {
            // бронь «захватывает» день, если пересекается с отрезком [день
00:00; следующий день 00:00)
            var dayStart = f.Date.Value.Date;
            var dayEnd = dayStart.AddDays(1);
            q = q.Where(b => b.StartAt < dayEnd && b.EndAt > dayStart);
        }

        // Текущие — ближайшие сверху; завершённые — последние сверху
        return f.Tab == "current"
```

```
            ? q.OrderBy(b => b.StartAt).ToList()
            : q.OrderByDescending(b => b.StartAt).ToList();
    }

    /// <summary>
    /// «Свободно ли оборудование с … по …»: для каждой единицы каталога —
свободна ли она
    /// и, если нет, почему (состояние или пересекающаяся бронь).
    /// </summary>
    public (string? Error, List<AvailabilityRow> Rows)
CheckAvailability(DateTime from, DateTime to)
    {
        if (to <= from)
            return ("Окончание периода должно быть позже начала", new());

        CompleteExpired();
        var equipment = _db.Equipment.AsNoTracking().OrderBy(e =>
e.Category).ThenBy(e => e.Name).ToList();
        var busy = _db.Bookings.AsNoTracking().Include(b => b.Employee)
            .Where(b => b.Status == BookingStatus.Active && from < b.EndAt && to
> b.StartAt)
            .ToList();

        var rows = equipment.Select(e =>
        {
            if (e.State != EquipmentState.Available)
                return new AvailabilityRow { Equipment = e, IsFree = false,
Reason = StateName(e.State) };
            var b = busy.Where(x => x.EquipmentId == e.Id).OrderBy(x =>
x.StartAt).FirstOrDefault();
            return b == null
                ? new AvailabilityRow { Equipment = e, IsFree = true }
                : new AvailabilityRow { Equipment = e, IsFree = false,
                    Reason = $"занято {b.StartAt:dd.MM HH:mm}–{b.EndAt:dd.MM
HH:mm} ({b.Employee?.FullName})" };
        }).ToList();
        return (null, rows);
    }

    /// <summary>Недельный график: какие часы каждого дня заняты у каждой
единицы оборудования.</summary>
    public List<WeekRow> WeekLoad(DateTime weekStart)
    {
        CompleteExpired();
        var start = weekStart.Date;
        var end = start.AddDays(7);
        var equipment = _db.Equipment.AsNoTracking().Where(e => e.State !=
EquipmentState.WrittenOff)
            .OrderBy(e => e.Category).ThenBy(e => e.Name).ToList();
        var bookings = _db.Bookings.AsNoTracking().Include(b => b.Employee)
            .Where(b => b.Status != BookingStatus.Cancelled && b.StartAt < end
&& b.EndAt > start)
            .OrderBy(b => b.StartAt).ToList();

        var rows = new List<WeekRow>();
        foreach (var e in equipment)
        {
            var row = new WeekRow { EquipmentId = e.Id, Title = $"{e.Name}
({e.InventoryNumber})" };
            foreach (var b in bookings.Where(b => b.EquipmentId == e.Id))
            {
                for (int d = 0; d < 7; d++)
                {
                    var dayStart = start.AddDays(d);
```

```
                    var dayEnd = dayStart.AddDays(1);
                    if (b.StartAt < dayEnd && b.EndAt > dayStart)
                    {
                        var s = b.StartAt < dayStart ? dayStart : b.StartAt;
                        var f = b.EndAt > dayEnd ? dayEnd : b.EndAt;
                        var surname = b.Employee?.FullName.Split(' ')[0];
                        row.Days[d].Add($"{s:HH:mm}–{(f == dayEnd ? "24:00" :
f.ToString("HH:mm"))} {surname}");
                    }
                }
            }
            rows.Add(row);
        }
        return rows;
    }

    public static string StateName(EquipmentState s) => s switch
    {
        EquipmentState.Available => "Доступно для бронирования",
        EquipmentState.Maintenance => "На обслуживании",
        _ => "Списано"
    };

    public static string StatusName(BookingStatus s) => s switch
    {
        BookingStatus.Active => "Активно",
        BookingStatus.Cancelled => "Отменено",
        _ => "Завершено"
    };
}
```

Приложение Г

Инструкция пользователя

Назначение. Модуль позволяет сотруднику найти свободное оборудование на нужное время и забронировать его, а ответственному – вести каталог и видеть загрузку оборудования.

Основные экраны: «Каталог» – оборудование с поиском и фильтрами; «Свободно ли?» – проверка доступности на период; «Забронировать» – форма новой брони; «Брони» – текущие и завершённые брони с фильтрами; «График» – загрузка по дням недели; «Сотрудники» – справочник.

Как забронировать: открыть «Свободно ли?», указать начало и окончание, нажать «Проверить», у нужного оборудования нажать «Забронировать на этот период», выбрать сотрудника, указать цель и нажать «Забронировать».

Как отменить: открыть бронь из списка «Брони», ввести причину и нажать «Отменить бронь». Отменить можно только активную бронь, которая ещё не закончилась.

Значение сообщений:

– «… уже забронировано с … по … (ФИО)» – выбранное время пересекается с другой бронью, выберите другое время или другое оборудование;

– «… нельзя забронировать: состояние „На обслуживании“» – оборудование в ремонте или списано;

– «Окончание брони должно быть позже начала», «Нельзя бронировать на время в прошлом», «Бронь не может быть длиннее 14 дней» – исправьте период;

– «Укажите цель использования», «Укажите причину отмены» – заполните поле;

– «Бронь уже отменена», «Нельзя отменить завершённую бронь» – бронь нельзя отменить повторно или после окончания;

– «У оборудования есть текущие брони» – перед выводом на обслуживание отмените их;

– «Запись не найдена (404)» – открыта ссылка на несуществующую запись.

Приложение Д

Инструкция запуска

Необходимые средства: .NET SDK 10.0 (dotnet --version), браузер, интернет при первой сборке.

Получение: распаковать архив или выполнить git clone https://github.com/Black287234/KP-Vit.git.

Запуск: dotnet run --project EquipmentBooking, затем открыть http://localhost:5090. В Visual Studio 2026 – открыть EquipmentBooking.sln и нажать F5.

База данных создаётся автоматически (миграция InitialCreate) в файле EquipmentBooking/booking.db вместе с учебными данными. Сброс – удалить booking.db при остановленном приложении. Альтернатива – dotnet ef database update --project EquipmentBooking.

Тесты: dotnet test – ожидается «Passed: 52, Failed: 0».

Проверка основного сценария: «Свободно ли?» на завтра 10:00–12:00 → забронировать документ-камеру; забронировать проектор INV-0001 на завтра 11:00–13:00 → отказ; отменить созданную бронь с причиной; открыть вкладки «Текущие» и «Завершённые и отменённые».